

Banking System

C++ Console Application

Pinnacle Labs Internship Task

Developer: Priyam Prakash

Language: C++ (C++11 Standard)

Date: May 2026

OOP File I/O Data Structures Serialization Encapsulation

1. Project Overview

This project is a **C++-based Banking Management System** that simulates real-world banking operations through a console interface. It allows users to create accounts, manage balances, perform deposits/withdrawals/transfers, and view complete transaction history.

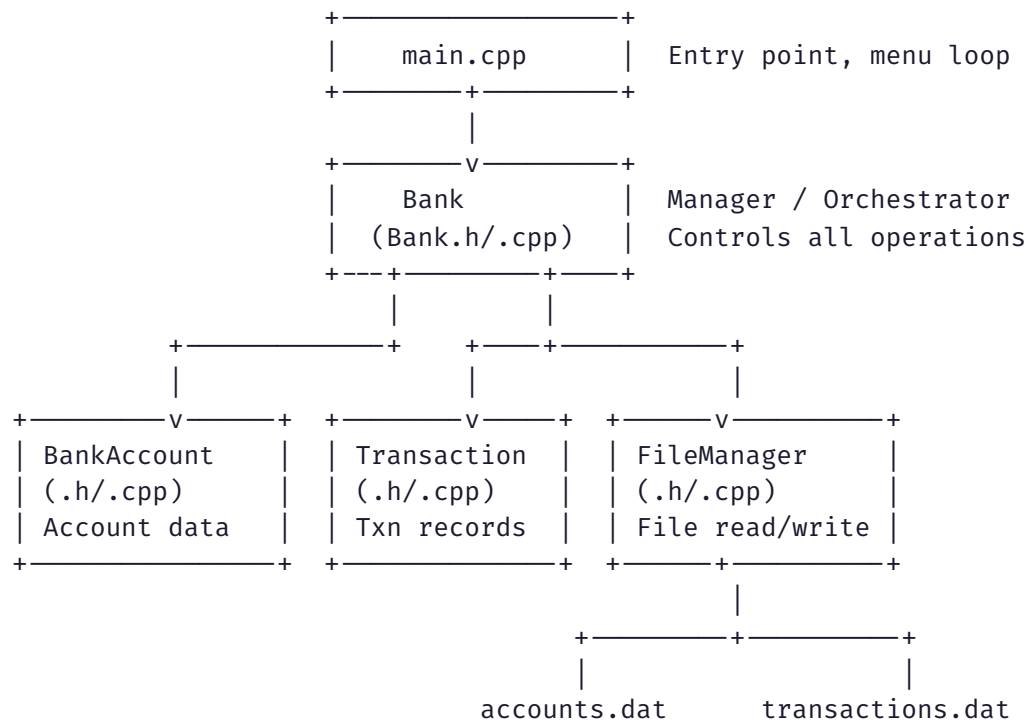
All data is **persisted to files**, meaning accounts and transactions survive between program restarts.

1.1 Features Implemented

#	Feature	Description
1	Create Account	Register with name, 4-digit PIN, minimum Rs. 500 deposit
2	View Account	Display account details after PIN verification
3	Deposit	Add funds to any active account
4	Withdraw	Remove funds with balance check and PIN auth
5	Transfer	Move money between two accounts
6	Balance Enquiry	Quick balance check with PIN
7	Transaction History	Full audit trail per account

8	List All Accounts	Tabular overview of all accounts
9	Close Account	Deactivate account, return balance

2. Architecture & File Structure



2.1 File Descriptions

File	Role
main.cpp	Program entry point. Displays the banner, runs the menu loop, routes user choices to Bank methods.
Bank.h / Bank.cpp	Central manager class. Orchestrates account creation, deposits, withdrawals, transfers, and history display.
BankAccount.h / BankAccount.cpp	Represents a single bank account with balance, PIN, and status. Handles deposit/withdraw logic.
Transaction.h / Transaction.cpp	Represents a single transaction record with type, amount, timestamp, and description.
FileManager.h / FileManager.cpp	Handles all file read/write operations. Saves/loads accounts and transactions to/from .dat files.

accounts.dat	Auto-generated data file storing all account records (pipe-delimited).
transactions.dat	Auto-generated data file storing all transaction records (pipe-delimited).

3. OOP Concepts Used

3.1 Encapsulation

All class data members are `private` . Access is provided through public getter methods and controlled operations.

```
class BankAccount {
private:
    std::string accountNumber; // Cannot be accessed directly
    std::string pin;           // Hidden from outside
    double balance;            // Modified only via deposit()/withdraw()
public:
    double getBalance() const; // Controlled read access
    bool deposit(double amount); // Controlled write access
};
```

Why? Prevents external code from setting invalid balances or bypassing PIN checks.

3.2 Abstraction

The `Bank` class hides the complexity of file I/O, account lookup, and transaction logging. The `main.cpp` simply calls `bank.deposit()` without knowing the internal details.

3.3 Modularity (Separation of Concerns)

Class	Responsibility
BankAccount	Manages one account's data and operations
Transaction	Represents and formats one transaction record
FileManager	Handles all file read/write (persistence layer)
Bank	Orchestrates everything (business logic layer)

main.cpp

User interface layer (menu and input)

3.4 Enum Class

```
enum class TransactionType {  
    DEPOSIT, WITHDRAWAL, TRANSFER_IN, TRANSFER_OUT, ACCOUNT_CREATION  
};
```

Scoped enum (`enum class`) is used instead of plain `enum` for type safety — prevents accidental integer comparisons.

4. Core Logic Explained

4.1 Account Creation Flow

1. User enters full name
2. User sets a 4-digit PIN (validated: must be exactly 4 digits)
3. User confirms PIN (must match)
4. User enters initial deposit (minimum Rs. 500)
5. System generates unique account number (e.g., PNB-1001)
6. Account is stored in `vector<BankAccount>`
7. Creation transaction is logged to `transactions.dat`
8. All accounts are saved to `accounts.dat`

4.2 Deposit Logic

```
bool BankAccount::deposit(double amount) {  
    if (amount ≤ 0) → reject (must be positive)  
    if (!active) → reject (account closed)  
    balance += amount;  
    return true;  
}
```

After a successful deposit, the `Bank` class creates a `Transaction` record and appends it to the file.

4.3 Withdrawal Logic

```
bool BankAccount::withdraw(double amount) {  
    if (amount ≤ 0) → reject  
    if (!active) → reject  
    if (amount > balance) → reject ("Insufficient balance!")  
    balance -= amount;  
    return true;  
}
```

Key protection: **You cannot withdraw more than the available balance.**

4.4 Transfer Logic

1. Verify sender account exists and is active
2. Verify sender PIN
3. Verify recipient account exists and is active
4. Check sender != recipient
5. Call `sender.withdraw(amount)`
6. Call `recipient.deposit(amount)`
7. Log TWO transactions: `TRANSFER_OUT` for sender, `TRANSFER_IN` for recipient

4.5 Account Number Generation

```
std::string Bank::generateAccountNumber() {  
    return "PNB-" + std::to_string(nextAccountId++);  
    // PNB-1001, PNB-1002, PNB-1003, ...  
}
```

On startup, the system scans existing accounts to find the highest ID and continues from there.

5. Data Persistence (File I/O)

5.1 Serialization Format

Data is stored as **pipe-delimited (|)** text in `.dat` files.

accounts.dat format:

```
PNB-1001|Priyam Prakash|1234|5000.00|1|2026-05-07 19:08:00
```

Fields: AccountNo | Name | PIN | Balance | Active(1/0) | CreationDate

transactions.dat format:

```
PNB-1001|DEPOSIT|2000.00|7000.00|2026-05-07 19:10:00|Cash deposit
```

Fields: AccountNo | Type | Amount | BalanceAfter | Timestamp | Description

5.2 Why Pipe-Delimited?

- Simple to parse with `std::getline(stream, field, '|')`
- Human-readable (you can open `.dat` files in Notepad)
- No external library needed (no JSON/XML parsers required)

5.3 File Operations

Operation	Method	Mode
Save all accounts	<code>saveAccounts()</code>	<code>ios::trunc</code> (overwrite)
Load all accounts	<code>loadAccounts()</code>	Read line by line
Log a transaction	<code>appendTransaction()</code>	<code>ios::app</code> (append)
Load transactions	<code>loadTransactions()</code>	Read line by line

6. Error Handling & Validations

Scenario	How It's Handled
Invalid menu input (letters)	<code>cin.fail()</code> check + <code>cin.clear()</code> to reset stream
Empty name	Rejected with error message
PIN not 4 digits	Loop until valid PIN entered
PIN mismatch on confirm	Loop until both match

Deposit below Rs. 500	Rejected with minimum amount message
Negative deposit/withdrawal	Rejected ("must be positive")
Withdraw more than balance	Rejected with available balance shown
Account not found	Error message returned
Wrong PIN	"Access denied" message
Operation on closed account	Rejected with appropriate message
Transfer to same account	Rejected
File doesn't exist (first run)	Gracefully returns empty vector

7. Key C++ Concepts Used

Concept	Where Used
Classes & Objects	BankAccount, Transaction, Bank, FileManager
Constructors (default + parameterized)	All classes
Enum class	TransactionType
Vectors (<code>std::vector</code>)	Storing accounts and transactions dynamically
String streams (<code>std::ostringstream</code>)	Serialization and formatting
File streams (<code>ifstream/ofstream</code>)	Reading/writing .dat files
Static methods	<code>Transaction::getCurrentTimestamp()</code> , deserialize methods
Const correctness	Getter methods marked <code>const</code>
<code>std::setw</code> , <code>std::setprecision</code>	Formatted table output
Header guards (<code>#ifndef</code>)	All .h files to prevent double-inclusion
<code>std::getline</code>	Reading full names with spaces, parsing pipe-delimited data

8. How to Compile & Run

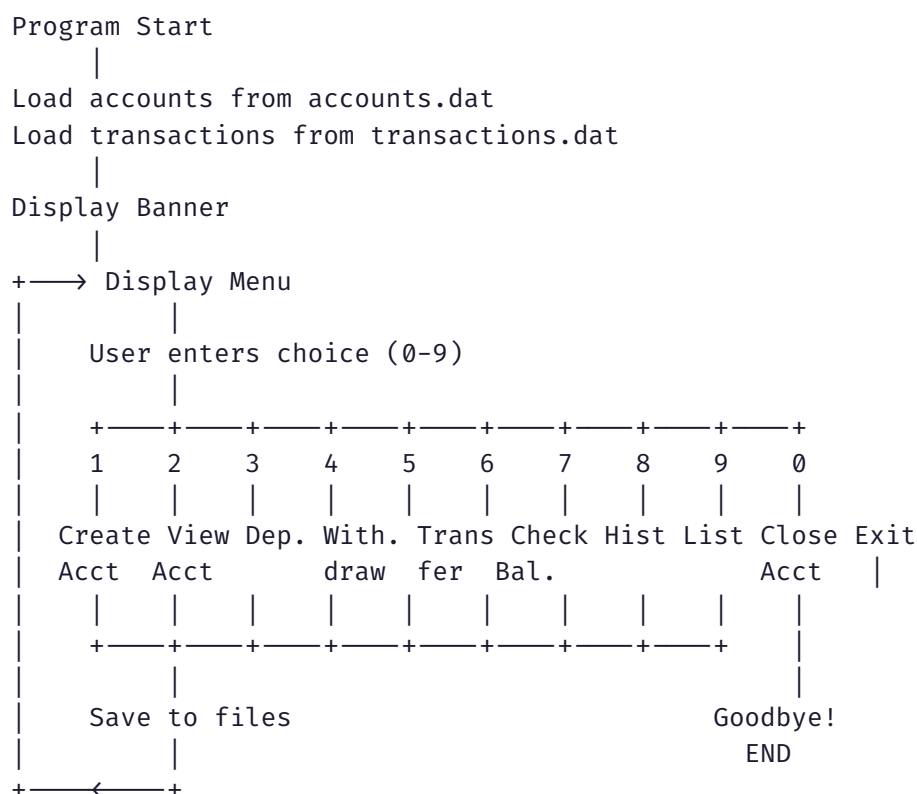
8.1 Using g++ (MinGW)

```
g++ -std=c++11 -o banking_system.exe main.cpp BankAccount.cpp Bank.cpp Transacti
banking_system.exe
```

8.2 Using Visual Studio Developer Prompt

```
cl /EHsc /std:c++17 main.cpp BankAccount.cpp Bank.cpp Transaction.cpp FileManage
banking_system.exe
```

9. Program Flow Diagram



10. Future Enhancements (Optional)

- Password hashing instead of storing plain PINs
- Database integration (SQLite) instead of flat files
- Interest calculation on savings accounts
- Multiple account types (Savings, Current)
- Admin login with different privileges
- Loan management module

Note: This project was developed as part of the Pinnacle Labs Internship Task. It demonstrates proficiency in C++ programming, Object-Oriented Design, File I/O, and building complete console applications.

Banking System v1.0 | Developed by Priyam Prakash | Pinnacle Labs Internship | May 2026